



U.S. General Services
Administration



Agentic AI Book Club - Session x

Reviewing Chapter 1.7B & 1.8

AI Community of Practice - 2026

Agenda

- Housekeeping
 - Chapter review
 - {{Other agenda items here}}
 - Quiz review
 - Intro to next week's chapters
- 

Housekeeping

- **Agentic AI is an early and rapidly evolving knowledge domain**
 - Contents of Chapter 1.8 should be re-inforced in other chapters/parts
 - No meeting on 5/5/2026 - Overall knowledge gap analysis result
 - Eval on 5/26/2026, next-day results. Cohort 2 Section 1 begins
 - Cohort 1 Section 2 starts on 6/3/2026 (2-3 chapters/week)
 - Part 3: Evaluation & Optimization (12 chapters)
 - Part 4: Production Deployment & Scaling (7 chapters)
 - Part 6: Retrieval-Augmented Generation (RAG) (9 chapters)
 - Part 7: NVIDIA NeMo Framework & Optimization (7 chapters)
 - Break weeks with invited industry SMEs
- 

Quiz Review

What is the fundamental limitation of vector-based retrieval systems when handling relational reasoning queries?

A:

- Vector embeddings dissolve relationships into the embeddings themselves, making multi-hop queries inefficient

Misconceptions to avoid:

- Iterative retrieval rounds can compensate for embedding-based relationship loss
 - Semantic similarity is a superset that includes relational similarity
 - LLM inference alone can reconstruct relationships from retrieved fragments reliably
- 

Quiz Review

How are multi-hop queries fundamentally different between vector RAG and knowledge graphs?

A:

- In knowledge graphs, multi-hop queries become graph traversal operations following explicit edges

Note:

- <https://aws.amazon.com/blogs/machine-learning/improving-retrieval-augmented-generation-accuracy-with-graphrag/>
- 

Quiz Review

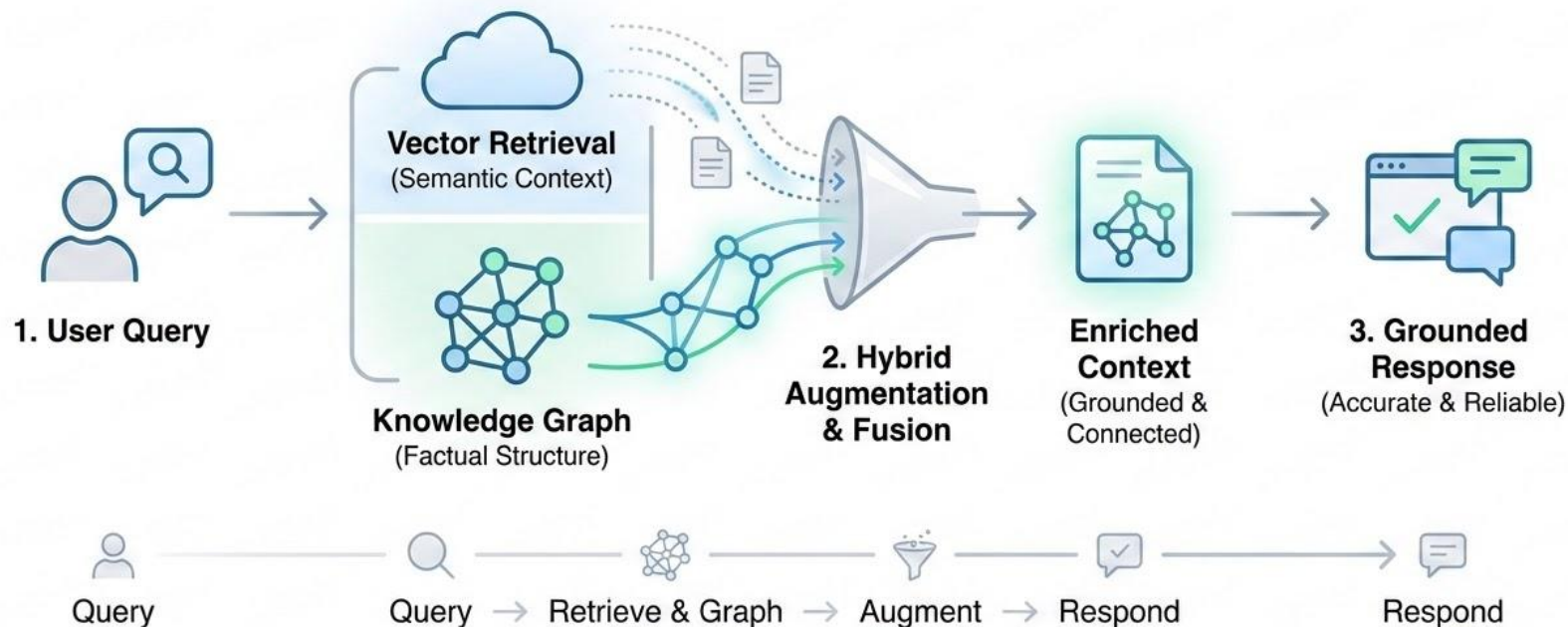
Your agent receives the query: "Find all venture capital firms that invested in companies that later went public through acquisitions by Fortune 500 companies." Which retrieval approach is most appropriate?

A: Knowledge graph - this requires explicit three-hop traversal (VC -> Company -> Acquisition -> Fortune500)

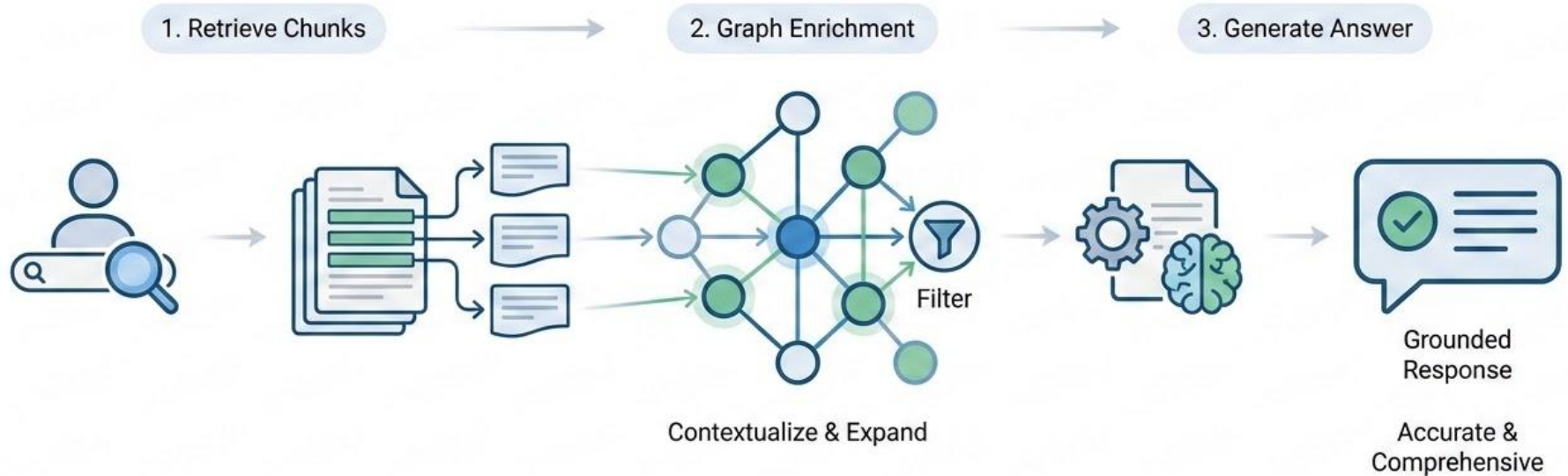
Note:

- Why RAG chain is not ideal - each reformulation step compounds retrieval noise, making the three-hop chain unreliable compared to deterministic graph traversal
 - Flawed trade-off analysis between velocity and accuracy - for this specific query type, RAG cannot achieve sufficient accuracy regardless of implementation quality
 - Flawed trade-off analysis between technologies - the three-hop traversal structure (VC -> Company -> Acquisition -> Fortune500) requires explicit relationship preservation, which vector embedding dissolves
- 

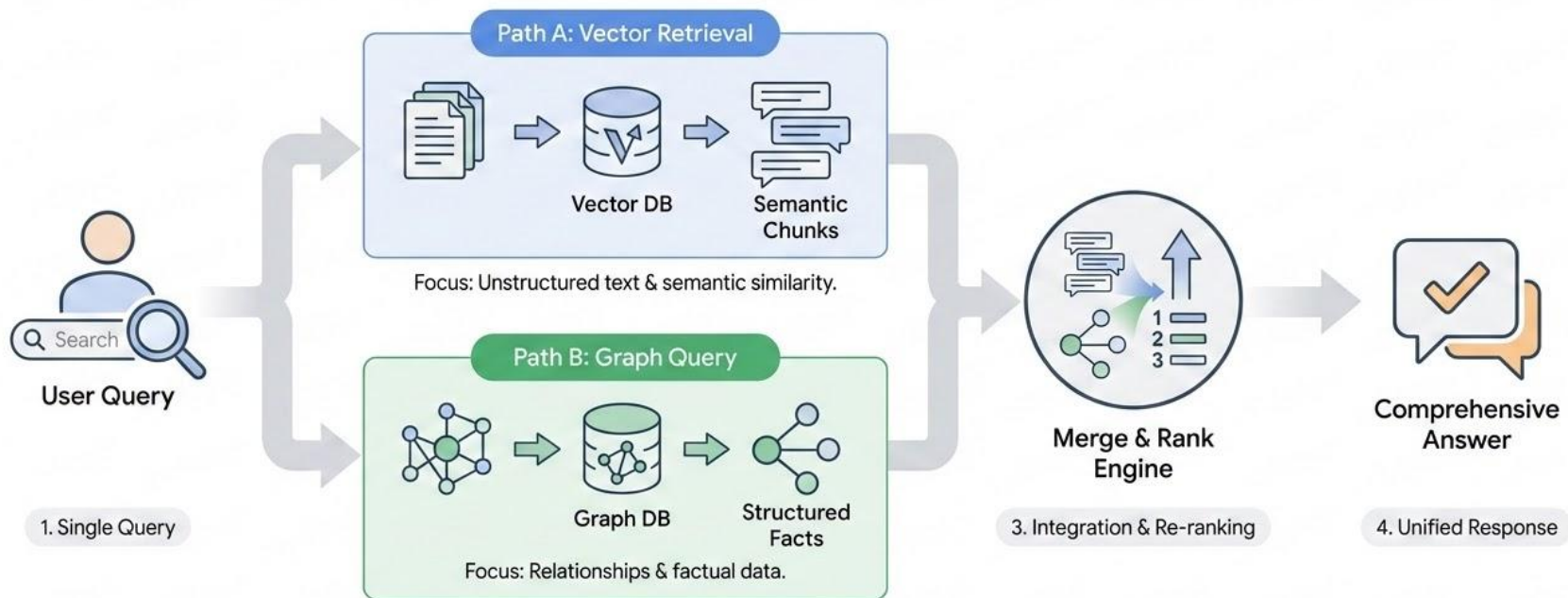
Hybrid Architecture - Retrieval-Augmented KG



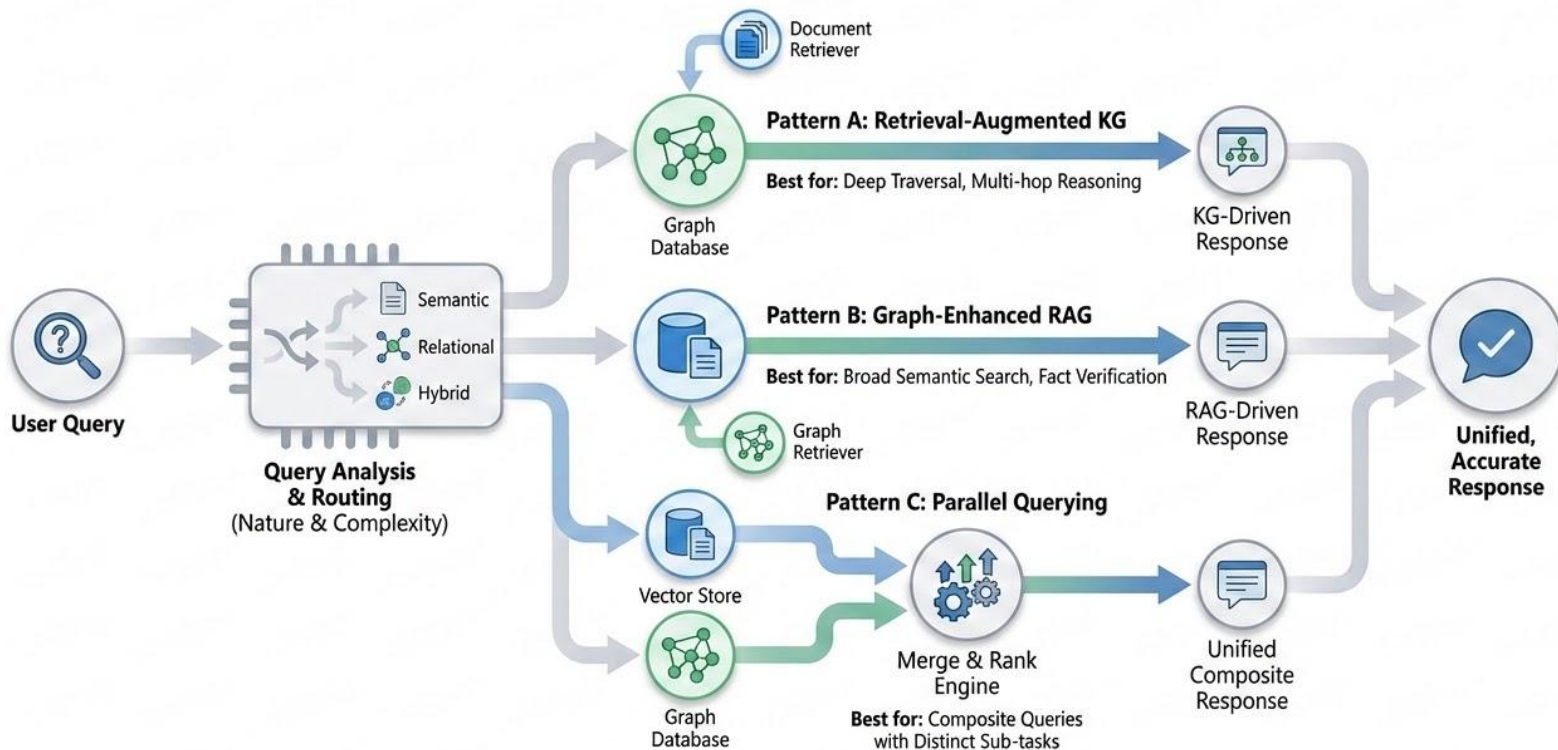
Hybrid Architecture - Graph-Enhanced RAG



Hybrid Architecture - Parallel Querying



Selecting Hybrid Architecture Patterns



Quiz Review

Your enterprise system receives 40% semantic queries ("What are best practices for microservices?"), 35% simple relational queries ("Who are the authors of paper X?"), and 25% complex multi-hop queries ("Which researchers co-authored with award winners and later moved to industry?"). Current infrastructure supports either vector RAG or knowledge graphs, but not both immediately. Development team can implement hybrid in 3 months. What architecture should you recommend?

A:

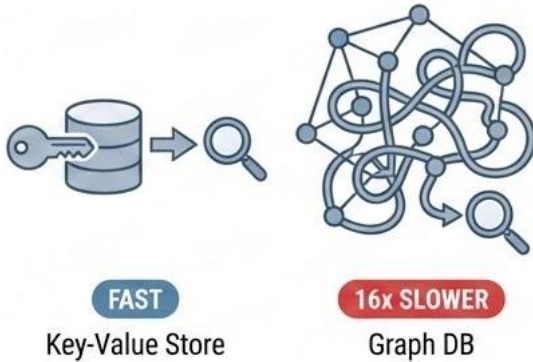
- Start with vector RAG now, add knowledge graph in 3 months for hybrid - serves 75% of queries immediately and addresses all query types eventually
- 

Hybrid Compliance System - The Worked Example

- Compliance scenario: detecting conflicts of interest
 - Vector store retrieves policy text and definitions
 - Knowledge graph traverses employment-family-investment chains
 - Entity extraction bridges unstructured text to graph nodes
 - Filter extracted entities to types modeled in your graph
- 
- A decorative graphic in the bottom right corner consisting of several overlapping, semi-transparent blue geometric shapes, including rectangles and triangles, arranged in a layered, ascending pattern.

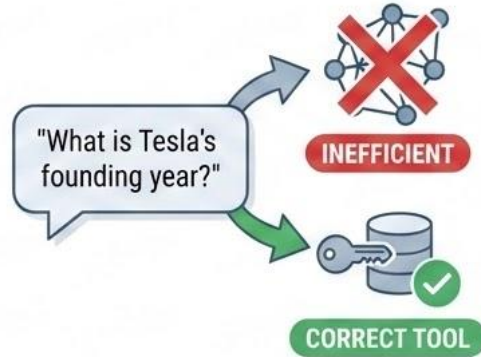
Pitfall - Graphs for Non-Relational Queries

SLOW DIRECT LOOKUPS



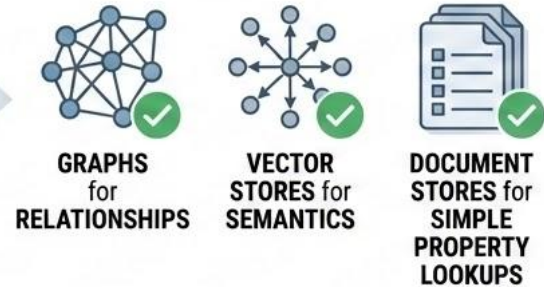
Direct property lookups are
16x slower in graph DB

SIMPLE FACTUAL QUERIES EXAMPLE



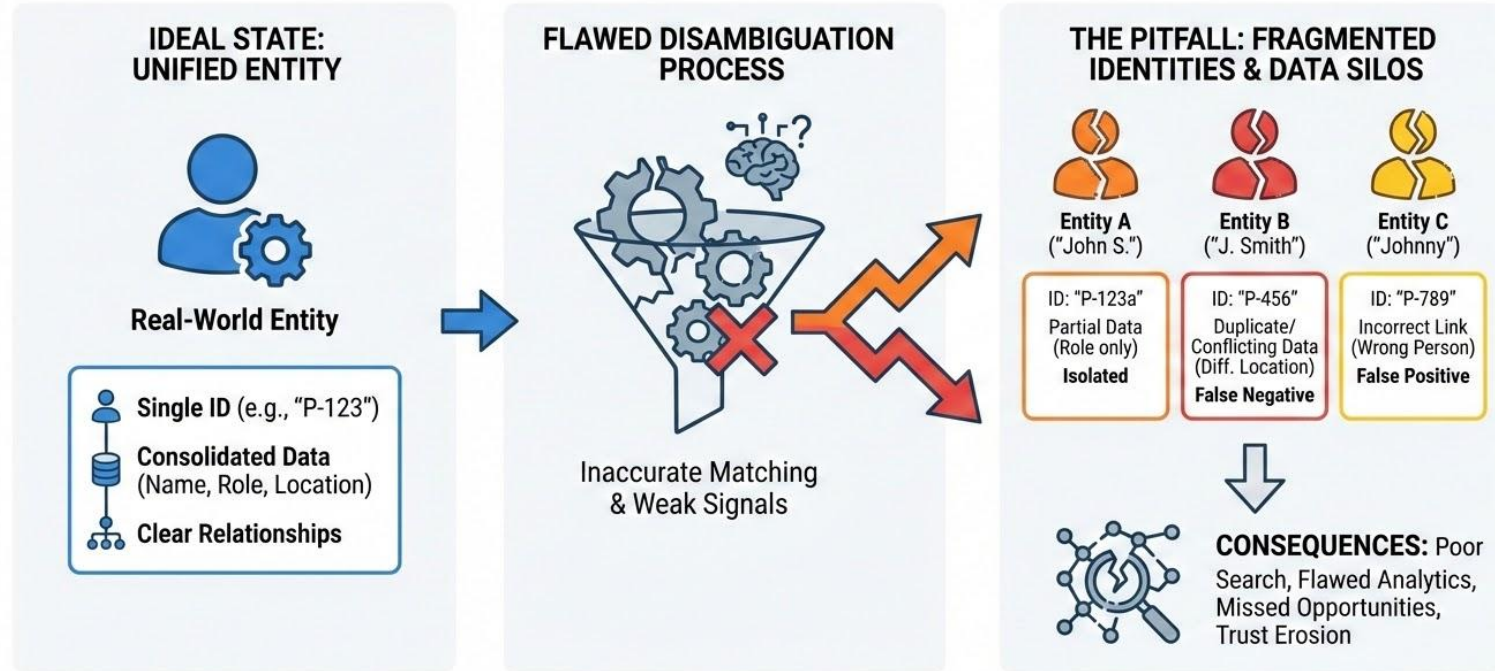
Needs key-value, not graph

RIGHT TOOL FOR THE JOB (SOLUTION)



Use graphs for relationships, vector
stores for semantics, document stores
for simple property lookups.

Pitfall - Entity Disambiguation Gone Wrong



Quiz Review

Your knowledge graph returns incorrect counts for "How many companies did Sequoia Capital invest in?" You discover the graph has 15 nodes for Sequoia Capital with slightly different names. What caused this and how do you fix it?

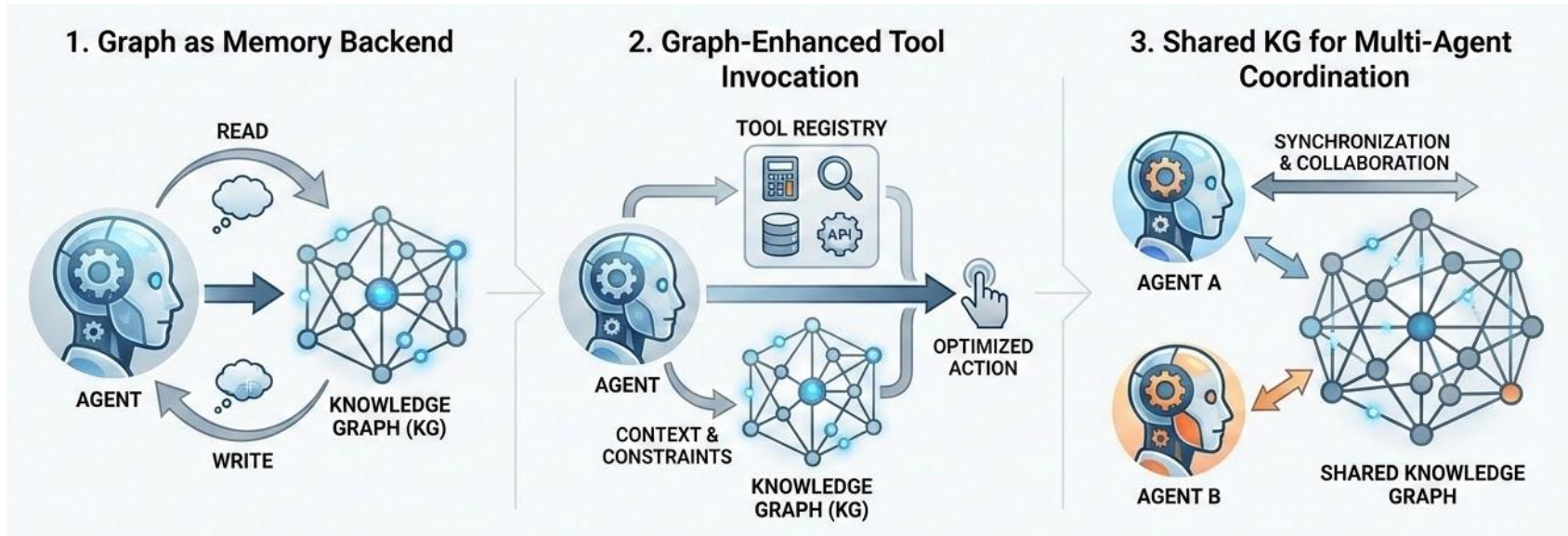
A:

- Missing entity linking - implement canonical identifier linking and consolidate duplicate nodes
- 

Pitfall - Cypher Queries That Explode

- Unbounded paths: `[\*]` explores every edge in the graph
- Scales exponentially with graph size
- Always bound path length: `[:KNOWS\*1..3]`
- Add indexes on frequently queried properties
- Profile with EXPLAIN to catch full node scans

Integration Patterns with Agent Architectures



Ref:

<https://neo4j.com/blog/developer/modeling-agent-memory/>

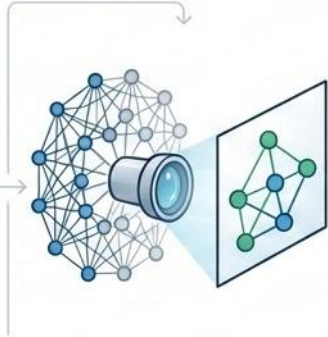
<https://dl.acm.org/doi/pdf/10.1145/3690624.3709238>

Production Optimization Strategies



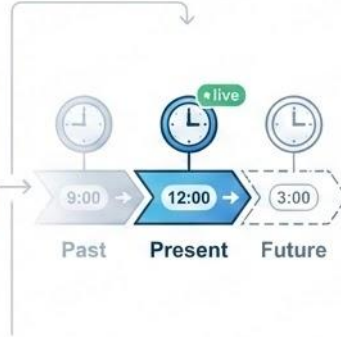
Index Optimization

Accelerate lookups with targeted indexing.



Graph Projections

Create focused views for specific analytic tasks.



Temporal Management

Efficiently handle time-series & versioned data.



Community Detection

Store cluster-IDs as precomputed node properties.



Retention Policies

Automate data aging to prevent unbound growth.

Worked Example

Your production knowledge graph was built from 100,000 documents processed in batches. Query "How many companies did Andreessen Horowitz invest in?" returns 47, but manual audit finds only 23. Multi-hop queries return incomplete results. Investigation shows:

- 47 nodes labeled "Company" with variations of "a16z", "Andreessen Horowitz", "Andreessen Horowitz LLC"
- Batch processing used CREATE commands
- Documents processed 2-3 times during development
- No entity linking step in pipeline

What is the root cause and what systematic fix is needed?

Worked Example

Your production knowledge graph was built from 100,000 documents processed in batches. Query "How many companies did Andreessen Horowitz invest in?" returns 47, but manual audit finds only 23. Multi-hop queries return incomplete results. Investigation shows: - 47 nodes labeled "Company" with variations of "a16z", "Andreessen Horowitz", "Andreessen Horowitz LLC" - Batch processing used CREATE commands - Documents processed 2-3 times during development - No entity linking step in pipeline What is the root cause and what systematic fix is needed?

WRONG CHOICE

Root cause: CREATE commands create duplicates on reprocessing.
Fix: Replace CREATE with MERGE on company names (MERGE (f:Firm {name: \$firm_name}))) and reprocess all documents — MERGE prevents future duplicates and consolidates existing ones during reprocessing

Note: MERGE does not work without entity linking

Worked Example

Your production knowledge graph was built from 100,000 documents processed in batches. Query "How many companies did Andreessen Horowitz invest in?" returns 47, but manual audit finds only 23. Multi-hop queries return incomplete results. Investigation shows: - 47 nodes labeled "Company" with variations of "a16z", "Andreessen Horowitz", "Andreessen Horowitz LLC" - Batch processing used CREATE commands - Documents processed 2-3 times during development - No entity linking step in pipeline What is the root cause and what systematic fix is needed?

WRONG CHOICE

Root cause: Documents were processed 2-3 times during development. Fix: Add a document processing registry tracking processed document IDs, and skip documents already processed — preventing reprocessing eliminates the duplicate creation mechanism

Note: single-pass processing can still create fragmentation without entity linking

Worked Example

Your production knowledge graph was built from 100,000 documents processed in batches. Query "How many companies did Andreessen Horowitz invest in?" returns 47, but manual audit finds only 23. Multi-hop queries return incomplete results. Investigation shows: - 47 nodes labeled "Company" with variations of "a16z", "Andreessen Horowitz", "Andreessen Horowitz LLC" - Batch processing used CREATE commands - Documents processed 2-3 times during development - No entity linking step in pipeline What is the root cause and what systematic fix is needed?

WRONG CHOICE

Root cause: Query layer doesn't handle entity name variations. Fix: Update all queries to use CONTAINS or fuzzy matching across name variations: WHERE firm.name CONTAINS 'Andreessen' OR firm.name CONTAINS 'a16z' — this handles existing data without expensive reprocessing

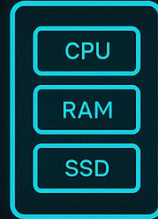
Worked Example

Your production knowledge graph was built from 100,000 documents processed in batches. Query "How many companies did Andreessen Horowitz invest in?" returns 47, but manual audit finds only 23. Multi-hop queries return incomplete results. Investigation shows: - 47 nodes labeled "Company" with variations of "a16z", "Andreessen Horowitz", "Andreessen Horowitz LLC" - Batch processing used CREATE commands - Documents processed 2-3 times during development - No entity linking step in pipeline What is the root cause and what systematic fix is needed?

CORRECT CHOICE

Root cause: Missing entity linking + CREATE creates duplicates. Fix: Add entity linking to canonical IDs + consolidate existing duplicates + switch to MERGE + reprocess with idempotent pipeline

Horizontal vs. Vertical Scaling



VERTICAL SCALING



More power in
one machine



Simpler to implement

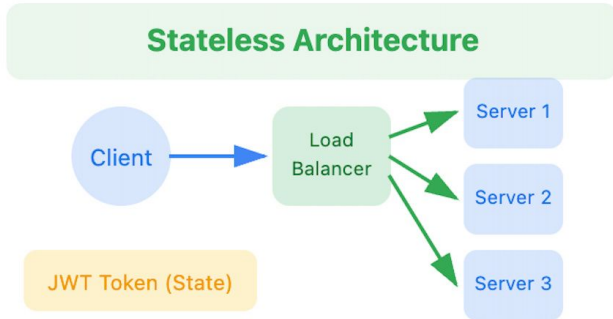
Hardware limits



HORIZONTAL SCALING

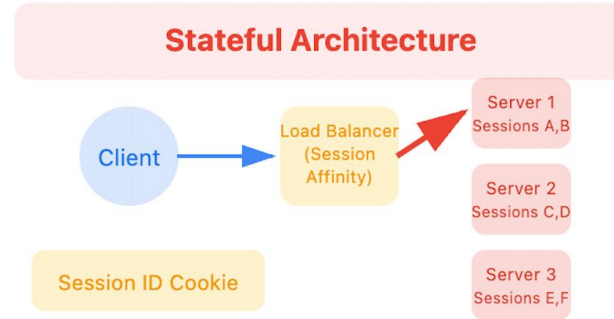
- Add more servers
- High availability
- Complex distributed systems
- Energy-per-task reporting

Stateless Architecture Enables Everything



Characteristics:

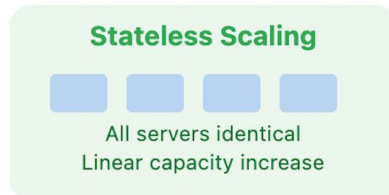
- ✓ No server memory
- ✓ Any server handles request
- ✓ Linear scaling



Characteristics:

- ✗ Server memory required
- ✗ Session affinity needed
- ✗ Complex scaling

Scaling Behavior Under Load



Queue-Based Autoscaling

- Traditional CPU metrics mislead for GPU workloads
- **Queue-to-compute ratio**: queue time divided by compute time
- Scale up at 800-1000 milliunits; scale down at 400-500
- Users care about wait time, not GPU utilization
- NVIDIA recommends this metric for NIM deployments

<https://developer.nvidia.com/blog/scaling-llms-with-nvidia-triton-and-nvidia-tensorrt-llm-using-kubernetes/>

A decorative graphic in the bottom right corner consisting of several overlapping blue rectangles of varying shades, creating a sense of depth and movement.

Asymmetric Scaling Behavior

- Scale up fast: 60-second stabilization window
 - Scale down slow: 5-minute stabilization window
 - Prevents oscillation and "thrashing" between states
 - New pods need 60-120 seconds for model loading
 - Over-provisioning briefly costs less than under-provisioning
- 

Next Chapters

- **Chapter 2.1: Framework Landscape**

This chapter provides a systematic decision framework for selecting among five major agent frameworks (LangGraph, LangChain, AutoGen, CrewAI, Semantic Kernel) by analyzing how their control flow models match workflow architectures, state management requirements, and collaboration patterns.

- **Chapter 2.2: LangGraph**

LangGraph is a framework for building agentic workflows through explicit graph architecture with nodes as computational units, edges as control flow pathways, and state as shared context. The chapter explores how graph-based design enables iterative refinement, conditional routing, and recovery from failures while establishing when LangGraph's sophistication is justified versus when simpler frameworks better match workflow requirements.